Chapter 2

AI Coding Tools

IN THIS CHAPTER

» **Flying with Copilot**

» **Coding with Tabnine**

» **Working together with Replit**

Generative AI has made new types of tools available to coders and has enabled many legacy tools to integrate AI functionality. In this chapter, you look at three of the most popular GenAI coding tools — GitHub Copilot, Tabnine, and Replit — gaining hands-on experience setting up and using the basic features of each.

Navigating GitHub Copilot

GitHub Copilot is a cloud-based AI coding tool developed by GitHub (which is part of Microsoft) and OpenAI (the creators of the GPT-x models behind ChatGPT and many other tools). Copilot was launched in June 2021 and currently integrates with several code editors and IDEs, including Visual Studio Code, Visual Studio, Neovim, and JetBrains's IDEs.

The GenAI model behind Copilot is named OpenAI Codex. Codex is based on OpenAI's GPT-3 and is also trained on source code from millions of public GitHub repositories and other publicly available source code.

Although Copilot works best with Python, JavaScript, TypeScript, Ruby, and Go, it has been trained on source code from more than a dozen programming languages and will continue to become more fluent in additional languages over time.

Installing the Copilot plug-in

The first step in using Copilot is to install the GitHub plug-in (or extension, as it's called in VS Code). The process for installing the extension differs based on your IDE. You can find detailed installation instructions for every IDE supported by Copilot at `https://docs.github.com/en/copilot/getting-started-with-github-copilot`. In Book 6, Chapter 1, I provide details on installing and enabling the Copilot extension for VS Code.

To follow along with the examples in this chapter, you should use VS Code or GitHub's in-browser code editor, CodeSpaces (`https://github.com/codespaces`). CodeSpaces is based on VS Code and, therefore, very similar to the installable version of VS Code.

When you install the Copilot extension in VS Code, the Copilot Chat extension is installed automatically. After you install Copilot, the Copilot status icon will appear in the lower-right corner of VS Code (see Figure 2-1, top). If you're not currently logged into a GitHub account with access to Copilot, the Copilot status icon appears with an exclamation point (see Figure 2-1, bottom).



Microsoft Corporation

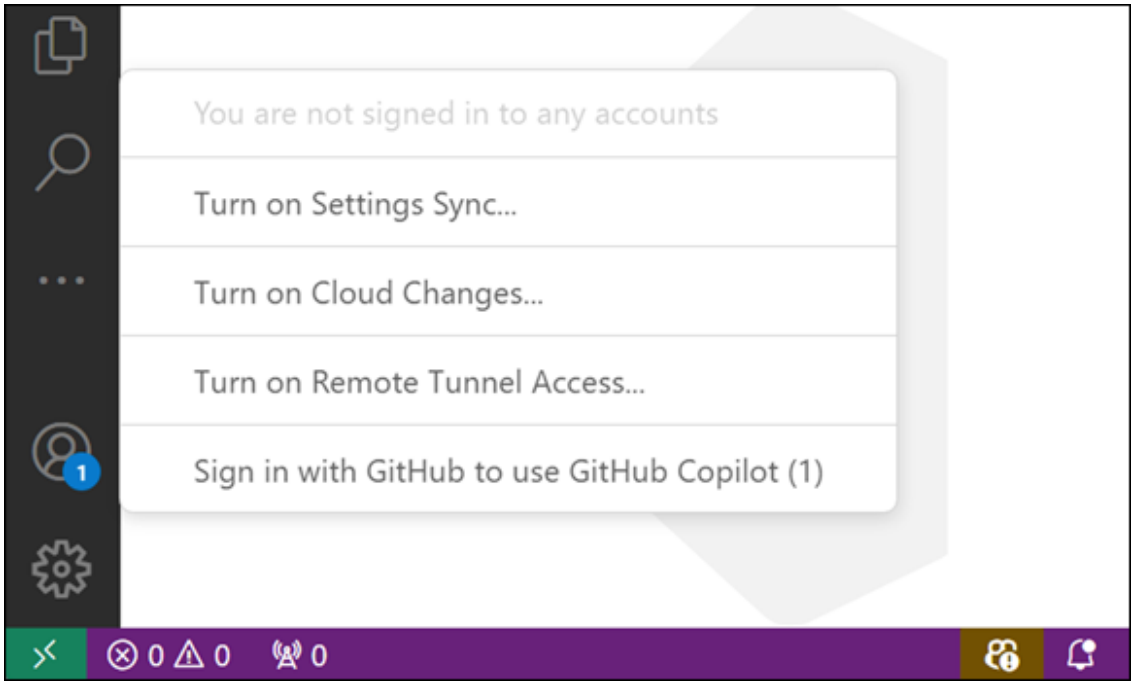
FIGURE 2-1: The Copilot status icon in connected and disconnected mode.

Although you can install the plug-in without having a Copilot account, you must have a GitHub account and either an individual subscription ($10 per month) or a business subscription ($19 per user per month) to

use Copilot. A free 30-day trial is available, as well as free accounts for students and educators.

You can click the Accounts icon in VS Code to create a GitHub account, if necessary. Then log into GitHub and grant the Copilot extension access, as shown in Figure 2-2.

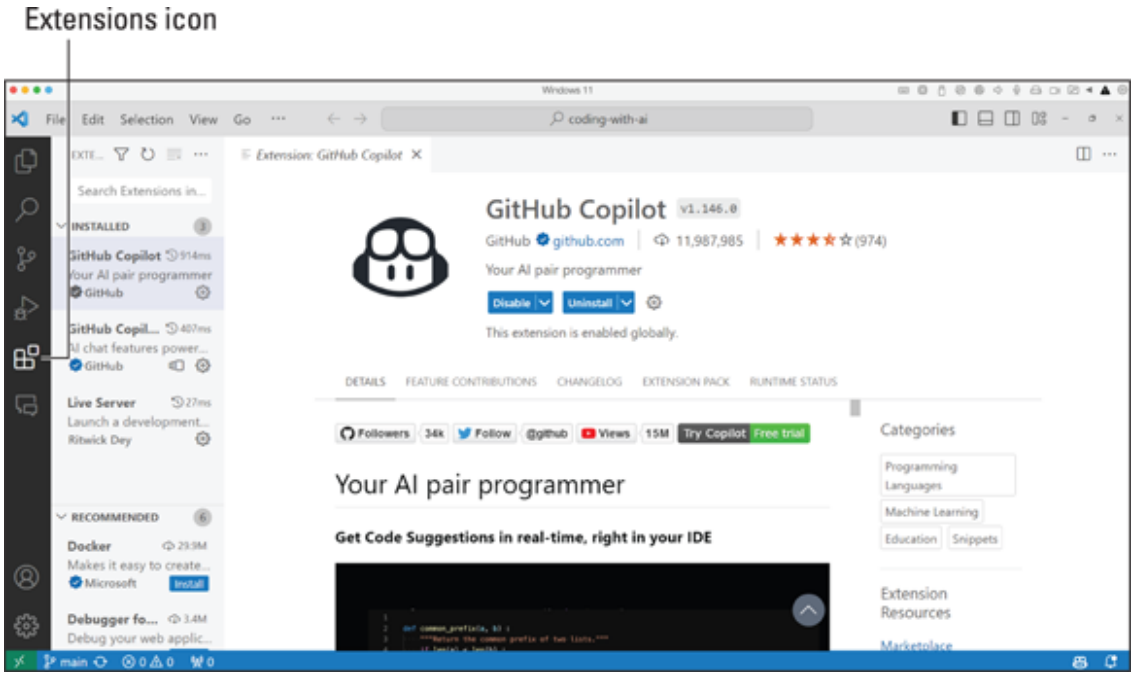
The Copilot extensions don't have many adjustable settings. Perhaps the most important setting you can control is whether or not Copilot is actively giving you suggestions. To disable Copilot so you can think, click the Copilot status icon. In the menu that appears at the top of the VS Code interface, select Disable Completions. (When completions are disabled, an Enable Completions link appears in the menu.)



Microsoft Corporation

FIGURE 2-2: Click the Accounts icon to grant Copilot access to your GitHub account.

To access other settings, click the Extensions icon in the left panel of VS Code. Then click the gear icon next to an extension and select Extension Settings from the menu that appears. Figure 2-3 shows the Extension Settings screen for the Copilot extension.



Microsoft Corporation

FIGURE 2-3: The Extension Settings screen for Copilot.

Working efficiently with Copilot

After you have installed and enabled Copilot, using it is just a matter of starting to type some code. However, just as with any GenAI tool, context is everything. The first piece of context you should provide to Copilot is the type of code you intend to write. You can do this by starting a new file with the file extension of the programming language you'll be using, such as .js for JavaScript or .py for Python.



TIP Choose your filename carefully because it determines the types of suggestions Copilot gives you.

Copilot generates suggestions using context from the file you're currently working on and neighboring tabs — the other files open in your editor. So, if your current project contains other files that will be related to the new one you're starting, open those as well so that Copilot can glean information from them.

The goal of learning how to provide good context to Copilot is to increase its accuracy. The longer you use Copilot on a project, the more its accura-

cy will improve on its own. You can further improve the accuracy of suggestions through better prompting and providing better context.



According to research published by GitHub, developers accepted an average of 30 percent of Copilot's suggestions immediately. After six months of using Copilot, the average suggestion acceptance rate increased to 34 percent. Reading this book may help you to beat the average!

Following are some best practices for getting the most accurate suggestions from Copilot:

- Specify your goal up front.
- Break things down into small steps.
- Give Copilot examples.
- Open related files.

You see examples of each of these best practices next.

Specify your goal

Especially if you're starting with a blank file or an empty repository, you'll want to begin to code by describing, in detail, what you want to create. Most often, you do this by writing code comments at the beginning of the file.

A vague comment is not going to result in Copilot being helpful. Here is the initial code generated by Copilot in response to the prompt *a to-do app:*

```
// a to-do app
var todoList = {
  todos: [],
  displayTodos: function () {
console.log('My Todos:', this.todos);
```

```
  }
};
```

Instead, try starting out with at least the amount of detail you would give to another coder:

```
/* A to-do app. Users can check a box to mark a task
complete, and completed tasks will be shown in a separate
list below the uncompleted tasks. */
var todoList = {
  todos: [],
  displayTodos: function () {
    console.log('My Todos:', this.todos);
  },
  addTodo: function (todoText) {
    this.todos.push({
      todoText: todoText,
      completed: false,
    });
    this.displayTodos();
  },
  changeTodo: function (position, todoText) {
    this.todos[position].todoText = todoText;
    this.displayTodos();
  },
  deleteTodo: function (position) {
    this.todos.splice(position, 1);
    this.displayTodos();
  },
  toggleCompleted: function (position) {
    var todo = this.todos[position];
    todo.completed = !todo.completed;
    this.displayTodos();
  },
};
```

With additional details, Copilot still didn't get it right, but at least it took into account some of the pieces of my prompt, such as that users can mark tasks as completed.

Break things down

GenAI models have a limited attention span. To improve results, give Copilot information as a series of steps, rather than as a request for a large block of code all at once. After each step, let Copilot generate some code, and then write the next step.

For example, in the following, I broke down the to-do app I want to write into areas of functionality, and I accepted the first suggestion Copilot made for each step.

```
/*
1. Create a constant called toDoItems
containing an array of objects. Each object
should have a todoText property and a completed property.
*/
const toDoItems = [
  {
    todoText: 'item1',
    completed: false,
  },
  {
    todoText: 'item2',
    completed: false,
  },
  {
    todoText: 'item3',
    completed: false,
  },
];
/*
2. Create a function that displays a list of to-do
items in an element with an id attribute set to "to-dos"
```

```
in the browser.
*/
function displayTodos() {
  const todosUl = document.querySelector('#to-dos');
  todosUl.innerHTML = '';
  toDoItems.forEach((item) => {
    const todoLi = document.createElement('li');
    todoLi.textContent = item.todoText;
    todosUl.appendChild(todoLi);
  });
}
```

This code is much closer to what I was looking for and is a decent starting point for my app.

Give examples

If you already have an idea of what you want Copilot to generate, start by giving it an example or two. For example, if you want it to generate an array of objects, tell it what an object should look like. In the following, I refined my first step from the preceding example to specify what I'm looking for:

```
/*
1. Create an array of to-do items with the following
shape:
[{todoTitle:'', todoDesc:'', date:'', completed:false}]
*/
const toDoItems = [
  {
    todoTitle: 'item1',
    todoDesc: 'item1 description',
    date: '2024-07-01',
    completed: false,
  },
  {
    todoTitle: 'item2',
```

```
    todoDesc: 'item2 description',
    date: '2024-07-02',
    completed: false,
  },
  {
    todoTitle: 'item3',
    todoDesc: 'item3 description',
    date: '2024-07-03',
    completed: false,
  },
];
```

Open related files

Copilot gets context for the file you're working on from other files you have open in your editor. If you're working on a JavaScript file that will affect an HTML page or that imports code from other JavaScript files, open those files in separate tabs.

In my to-do app example, the JavaScript will affect the display of the HTML page, so creating the HTML page and opening it in another tab will help Copilot generate more accurate JavaScript code.

Using keyboard shortcuts

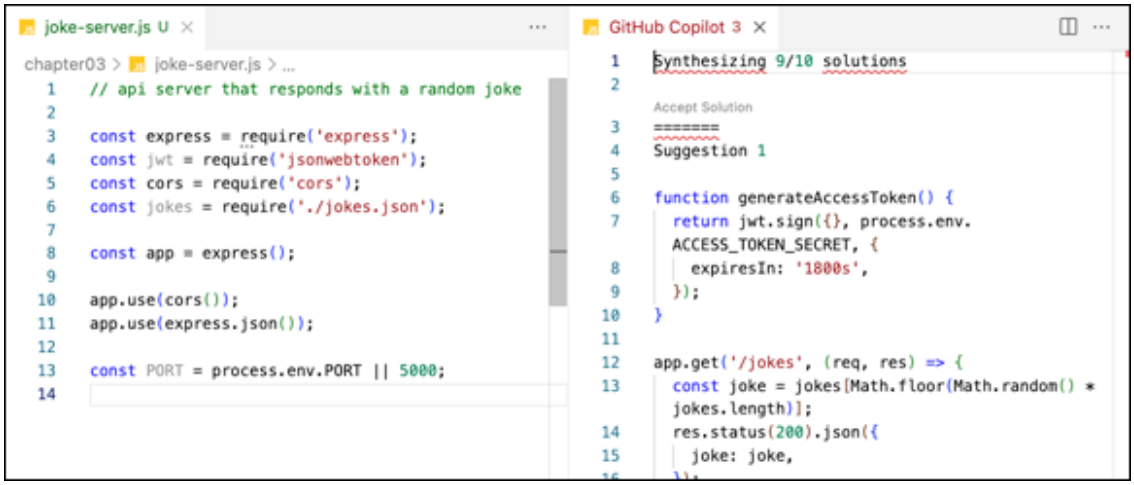
Although you can successfully use Copilot by just coding as you normally would and using the tab key to accept suggestions, knowing a few more keyboard shortcuts will make your coding sessions more productive. The keyboard shortcuts you'll want to remember are shown in Table 2-1.

TABLE 2-1 Copilot Keyboard Shortcuts

Shortcut	*What It Does*
Tab	Accepts inline code suggestions
Esc	Dismisses inline code suggestions

Shortcut	*What It Does*
Alt +] (or Option +])	Shows the next suggestion
Alt + [(or Option + [)	Shows the previous suggestion
Alt + \ (or Option + \)	Triggers a suggestion
Ctrl + Enter	Generates up to 10 suggestions in a separate pane

The Ctrl+Enter shortcut is particularly useful. When Copilot gives you a suggestion and you want to find out whether it might be able to suggest something better, press Ctrl+Enter. A new pane will open in your code editor and (after a minute or so) display up to 10 other suggestions, as shown in Figure 2-4.



Microsoft Corporation

FIGURE 2-4: Getting more suggestions.

Exploring Tabnine

Tabnine is an AI coding assistant and the name of the company that created it. The company was formed in 2012 and released the first version of its coding assistant in 2018, making Tabnine one of the longest-established AI-powered coding tools.

Tabnine is available in three versions: starter, pro, and enterprise. The starter version offers basic code completion. The pro plan offers AI code features similar to those of Copilot: whole-line and function code completions, natural-language-to-code completions, and a chat interface. The enterprise version allows a company to locate the model in the corporate firewall and provide access to every developer in the company.

A 7-day free trial (which for me turned out to be a 14-day trial) of the pro plan is available. I show you features from both the starter plan and the pro plan in this chapter.

One of the biggest differences between Tabnine and other AI coding assistant tools is that Tabnine runs on its own model, which they've trained on publicly available open-source code with permissive licenses. So coders and organizations should be able to use Tabnine-generated code without facing legal issues regarding intellectual property violations. This added assurance has made Tabnine popular with large organizations.



TECHNICAL STUFF You can view the entire list of repositories that Tabnine is trained on by going to `https://trust.tabnine.com/`.



WARNING Be aware of the risk of violating the licenses of the code a model was trained on. In 2022, a class action lawsuit was filed against Microsoft (the owner of GitHub and GitHub Copilot) and OpenAI, claiming that the companies violated the open-source licenses of programmers whose code was used to train the model Copilot uses.

Tabnine takes code privacy seriously. Their privacy policy (available at `https://tabnine.com/code-privacy`) states that they never store or share any of your code. This point is important for businesses concerned that AI coding assistants may inadvertently leak their intellectual property.

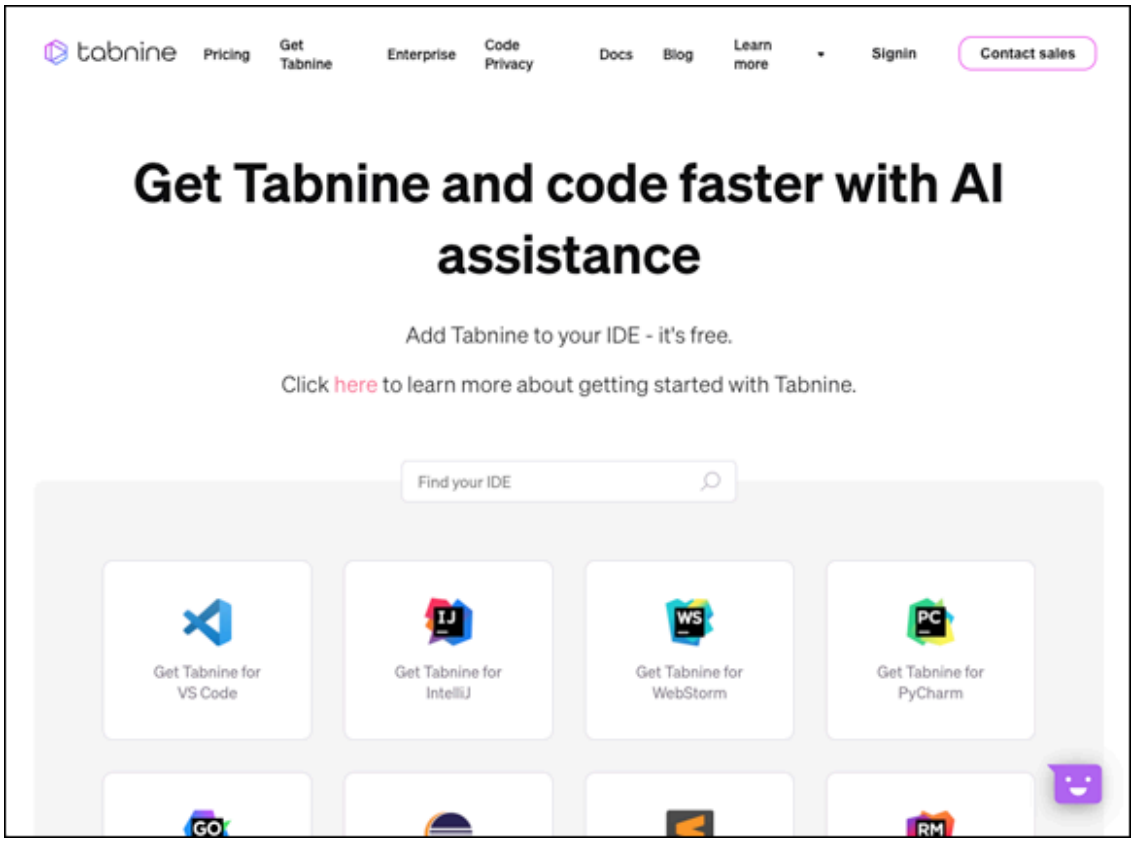
Businesses that use Tabnine can create their own model, which can live on the business's premises or in the cloud. To create a private code model, Tabnine trains its public model with the client's codebase. When customers with a private model submit a query, it's sent to Tabnine's public model and the private model and picks the most relevant code suggestion from the two options returned. The private model is continuously trained from the code and decisions made by its users.

Installing Tabnine

Tabnine is available as a plug-in, or extension, for many of the most popular code editors, including VS Code, Eclipse, Android Studio, WebStorm (as well as all JetBrains code editors), and Sublime.

Follow these steps to install Tabnine:

1. **Go to `https://www.tabnine.com` and click the Get Tabnine link at the top of the page.**
   A page with a Search box and a list of IDEs appears, as shown in Figure 2-5.
2. **Select your IDE and follow the steps to install the extension.**



Tabnine

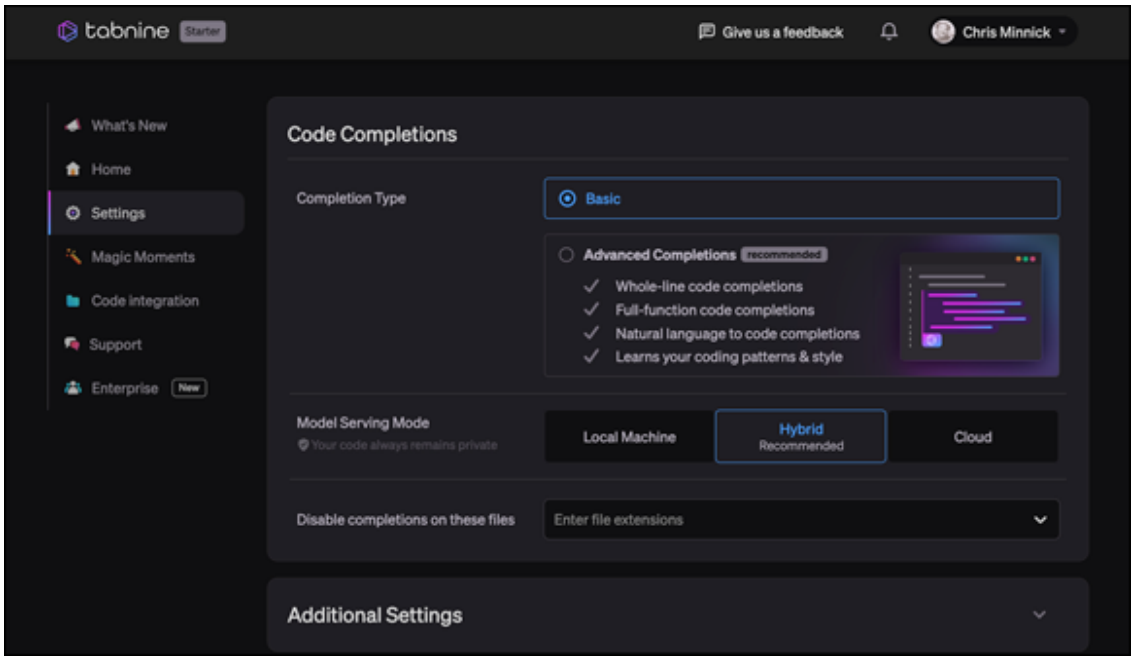
FIGURE 2-5: Choosing the IDE where you want to install the Tabnine extension.

Before you can use the Tabnine extension, you need to create a Tabnine account by going to `https://app.tabnine.com/signup`. After you've installed the Tabnine extension, you may need to restart your IDE.

Setting up Tabnine

After you've installed the extension and logged in, you'll see a link at the bottom of your IDE (in the case of VS Code) that says what plan you're subscribed to. Click this link to open Tabnine Hub.

Tabnine Hub is where you can adjust settings and read about the latest features. If you're using the starter plan, as I am in Figure 2-6, you'll also see links for a lot of features that you won't have access to and links to upgrade to a paid plan. The starter plan is quite capable, however, and you may want to stick with it while you learn about Tabnine's capabilities.



Tabnine

FIGURE 2-6: Tabnine Hub.

Tabnine allows you to choose from one of three model-serving modes: local machine, cloud, or hybrid. Which of these you choose will affect the quality of responses you get:

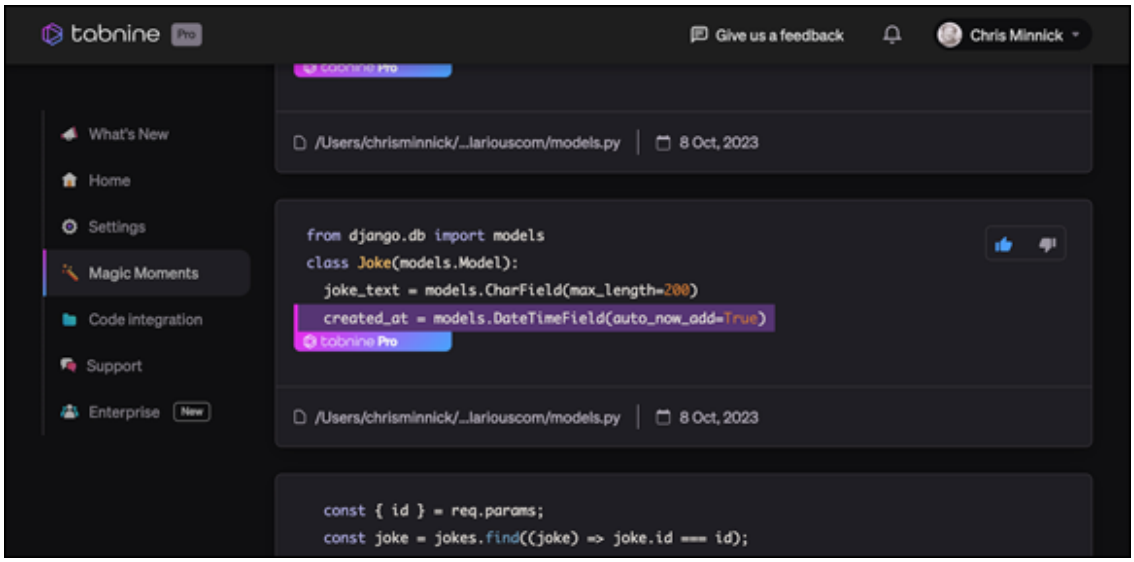
- **Local machine:** When you choose the local machine mode, Tabnine stores a smaller copy of the model on your local machine and allows you to work offline without any of your code being sent to Tabnine's servers. The local model is not as powerful as the cloud model, however, and it will increase your local machine's CPU and memory usage.
- **Cloud:** The cloud model uses Tabnine's servers, so it requires a connection to the internet. Because it processes your completions on the server, the results you get from the cloud model will be more accurate and longer.
- **Hybrid:** The hybrid model, which is the default mode, combines the benefits of both the cloud and local machine models. You can get suggestions while offline but also take advantage of the computing power of the cloud.

None of the three models stores any of your code. The cloud modes use a technique Tabnine calls *ephemeral processing*, in which your code is processed by the model and then immediately discarded after the model returns a completion.

Understanding Tabnine's AI-driven code completion

To get started coding with Tabnine, you work in much the same way as you would if you were using Copilot. As you enter code in your editor, Tabnine makes suggestions. You accept the suggestions by pressing the tab key.

To help the local model learn about your coding style, you can rate Tabnine's suggestions. Open Tabnine Hub and click Magic Moments in the left navigation, as shown in Figure 2-7. Magic Moments is available only in the Pro version (or the free trial). On the Magic Moments screen, give a thumbs up or thumbs down to individual suggestions. Tabnine uses this feedback to help the local model learn about your coding style.



Tabnine

FIGURE 2-7: Viewing previous "magic moments."

Like other AI code assistant tools, Tabnine requires context to be able to give good suggestions. Here are some tips for improving the suggestions you get from Tabnine:

- Write more comments in your code than you normally would.
- When you start working with Tabnine, act as if it's a junior developer who's new to the project. Don't assume that Tabnine can guess what you want it to do, even if it seems obvious to you.
- Be patient. Since Tabnine learns from previous code of yours that it's seen, it may take time for the model to learn your preferences.

Reviewing Replit

Replit is a browser-based IDE with AI assistance features, collaboration features, and a large and active community of developers. Replit has support for every popular programming language and framework. Because it's an in-browser environment, using it doesn't require any setup beyond creating a free account.



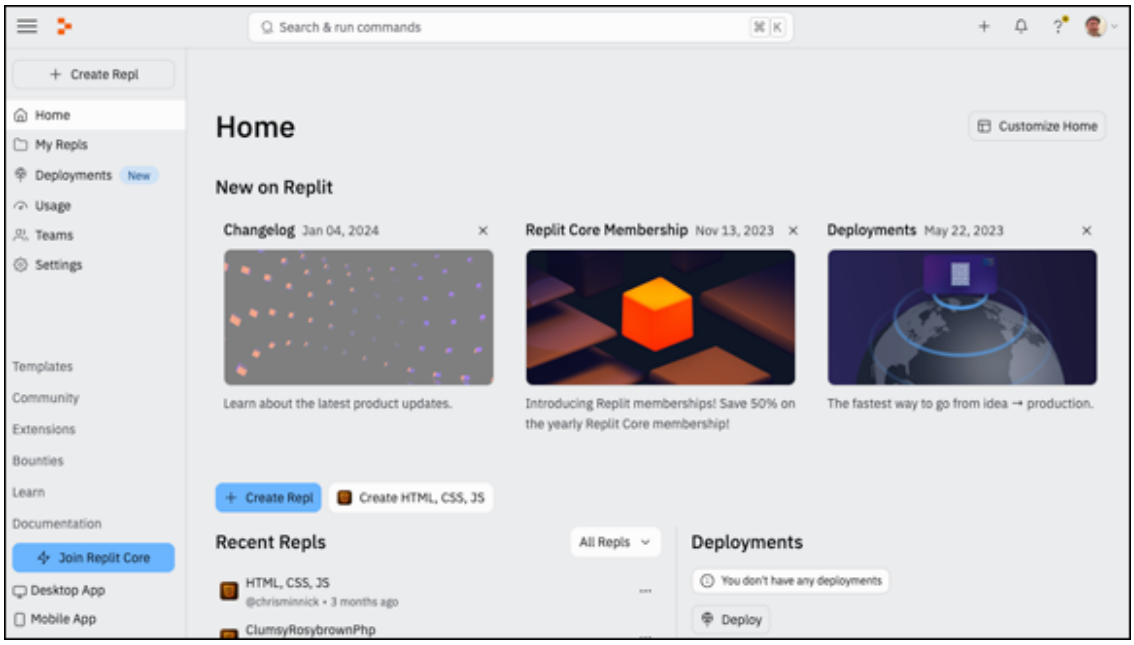
TIP After you've created a program using Replit, you can then use it for deploying the program. If you prefer to work offline or be free from the distractions of the Replit website, download and install the Replit desktop or mobile app.

To get started with Replit, go to `https://replit.com` and create an account using your email address, a Google account, or your GitHub account. The default home page is shown in Figure 2-8.

Although the core feature of `Replit.com` is the IDE, the site also has a Learn section where you can view video tutorials and interactive coding lessons.

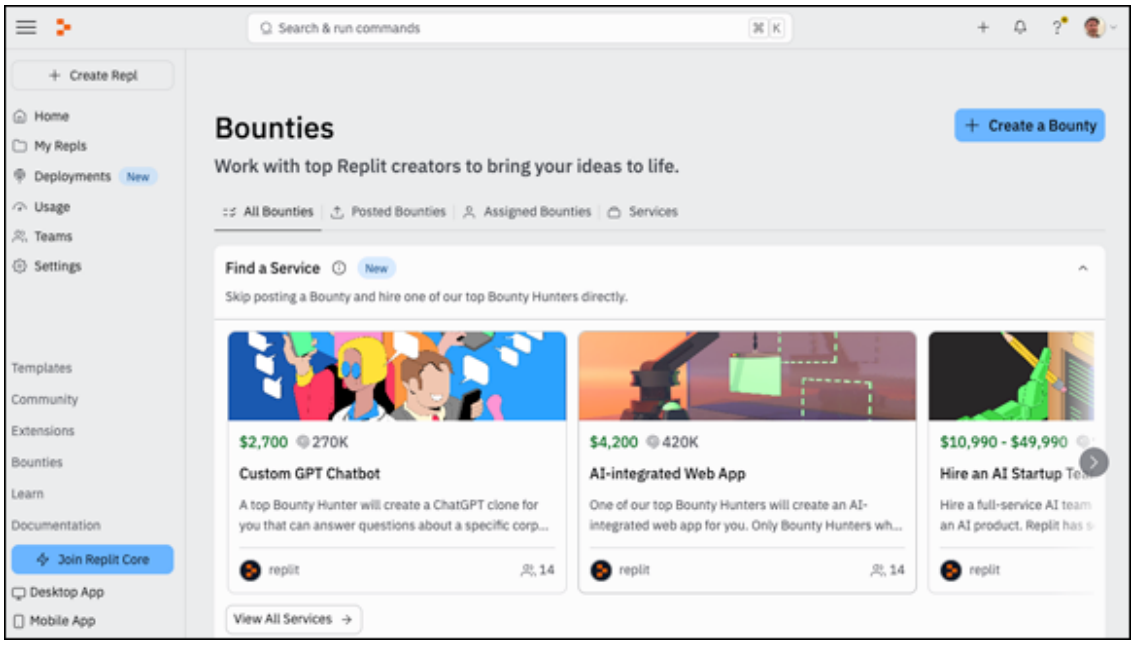
In the Bounties section of Replit, shown in Figure 2-9, people and companies post programming jobs, and coders advertise their services, along with a price for each.

The fastest way to get started on a project in Replit is by using one of the prebuilt templates, which are available by clicking the Templates link in the left navigation bar. On the Templates page, shown in Figure 2-10, you can find a template by searching or browsing.



Replit, Inc.

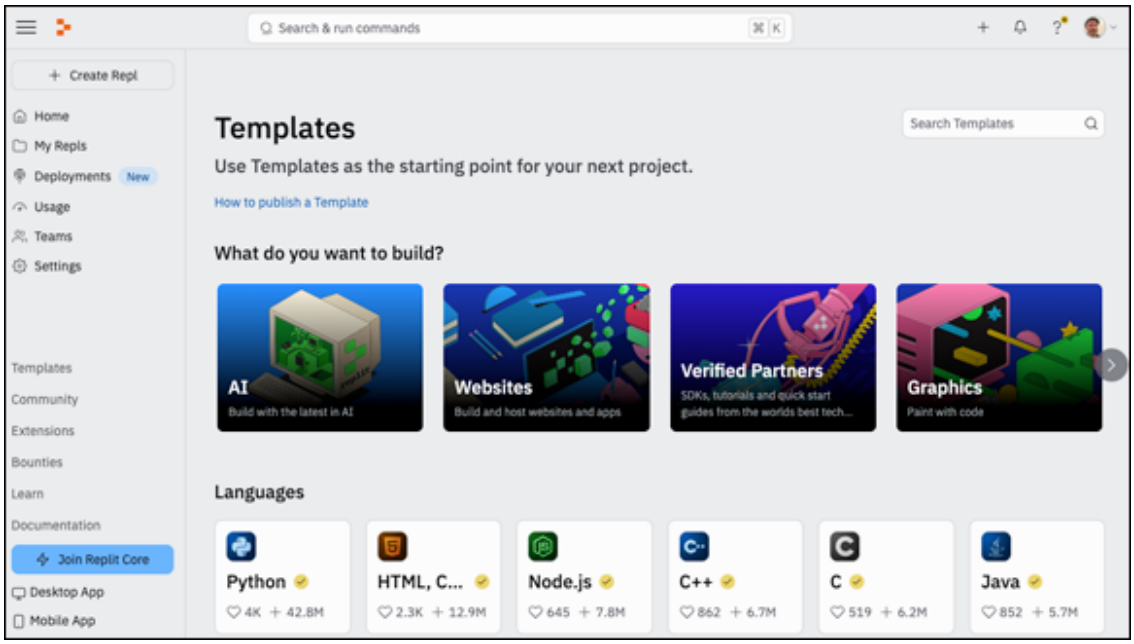
FIGURE 2-8: The Replit home page.



Replit, Inc.

FIGURE 2-9: Coders can advertise and find gigs through Replit Bounties.

If you prefer to start from scratch, you can create a project without using a template by clicking the Create Repl button on the home page or in the left navigation bar.



Replit, Inc.

FIGURE 2-10: Get started quickly with a template.

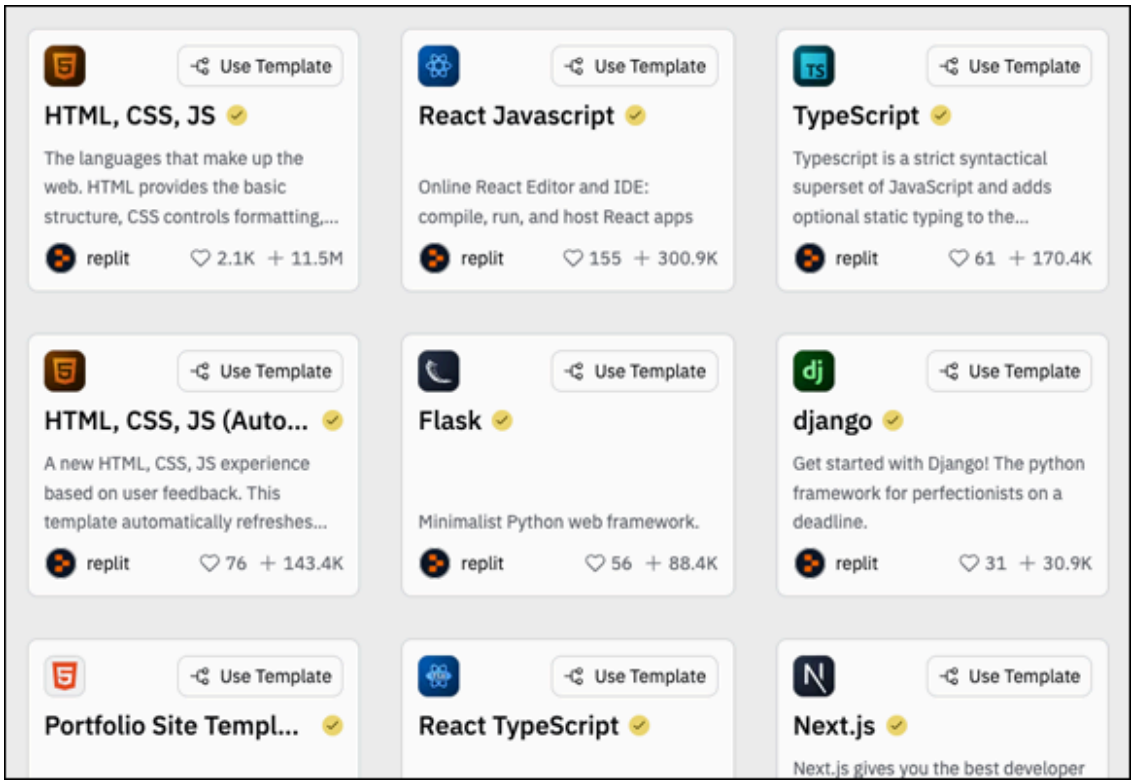
Starting a website with Replit

It's time to experiment with Replit and its AI features. Follow these steps to build a website using HTML, CSS, and JavaScript:

1. **While logged in to Replit, click the Templates link in the left navigation.**
2. **Click the Websites link and then locate the HTML, CSS, JS template.**
   You can also find the template by using the search bar.

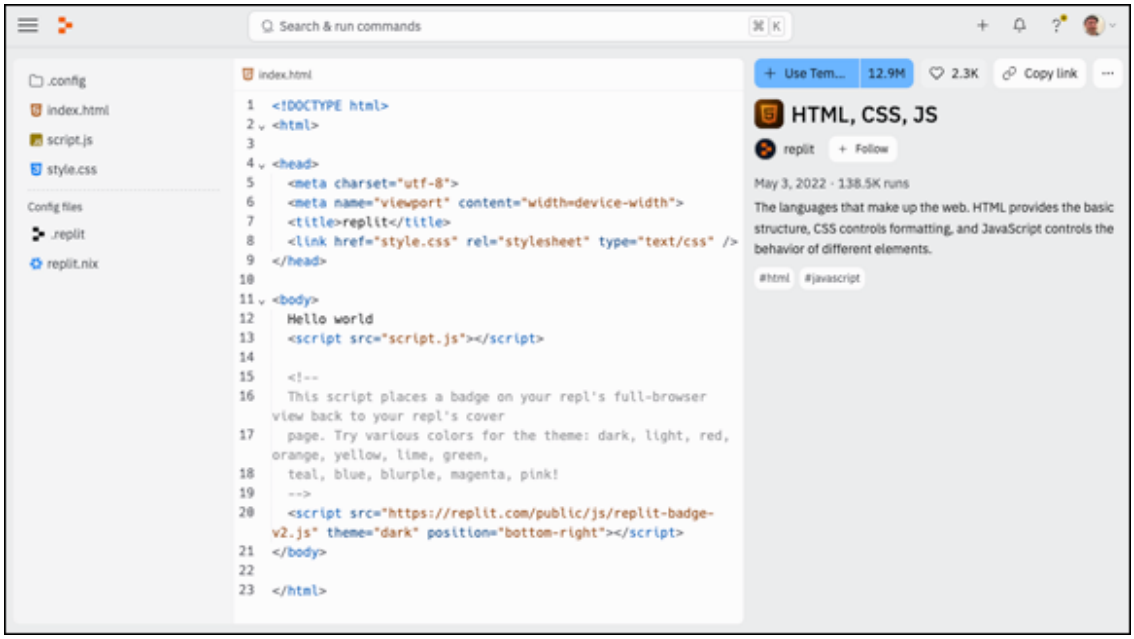
   

   TIP When you're searching for a template and several are available, choose the most popular one by looking at the icons that indicate the number of likes and runs, as shown in Figure 2-11.
3. **Click in the box for the HTML, CSS, JS template, but don't click the Use Template option yet.**
   The template appears, as shown in Figure 2-12.
4. **Read through the description of the template and click each of the files in the left panel (index.html, script.js, and style.css) to see what they do.**
   In the case of the HTML, CSS, JS template, there's not much to see except that it displays the text *Hello World.*
5. **Click Use Template and give your website a name and description in the pop-up box that appears.**
6. **Still in the pop-up window, click Use Template again.**
   The template opens in the Replit workspace, as shown in Figure 2-13.



Replit, Inc.

FIGURE 2-11: A template's popularity is often a good indicator of its quality and usefulness.



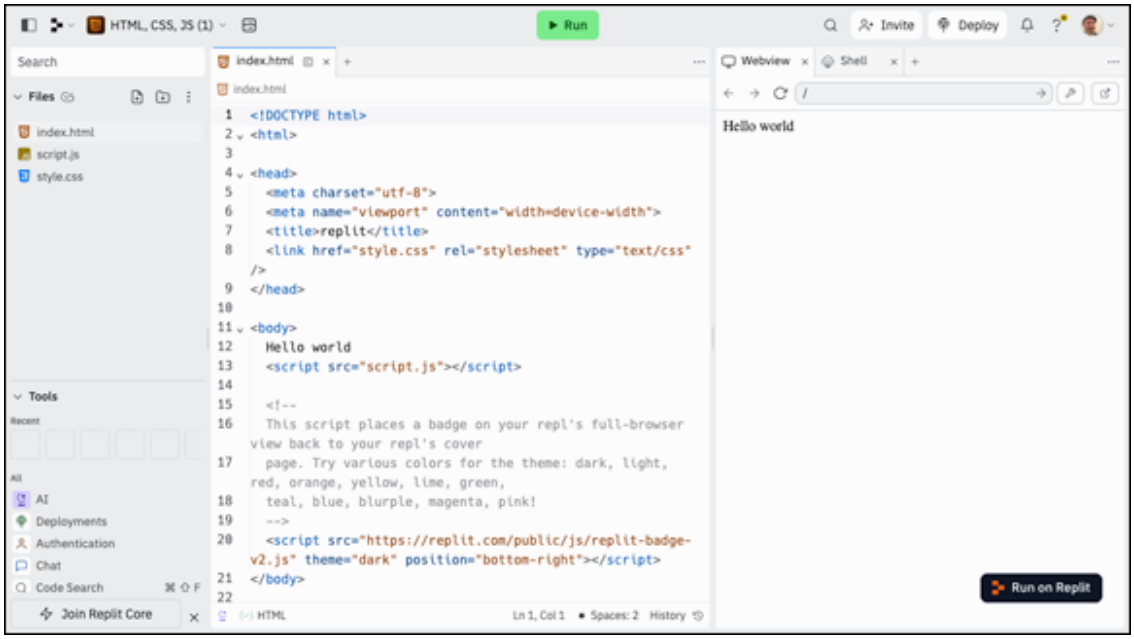
Replit, Inc.

FIGURE 2-12: Viewing more information about a template.

Exploring the Replit workspace

The Replit workspace is an in-browser IDE that includes features for creating, debugging, and collaborating on software. The workspace is split

into two areas: the sidebar area and the tabs and panes area.



Replit, Inc.

FIGURE 2-13: Your copy of the template in the Replit workspace.

The sidebar

The sidebar is the left column of the workspace. At the top of the sidebar is the file explorer. Here is where you can organize and create files and folders in your project. The file explorer works the same way as most file browsers. Clicking a file in the list opens that file for editing.

Below the file explorer are the tools, including a debugger, a command shell, a web browser (called Webview), and an AI assistant tool (which you learn about in the next section). To see all the tools, you may need to resize the Tools panel. Figure 2-14 shows all the tools that are currently part of the workspace.

Clicking a tool opens it in a new tab to the right of any open code files.

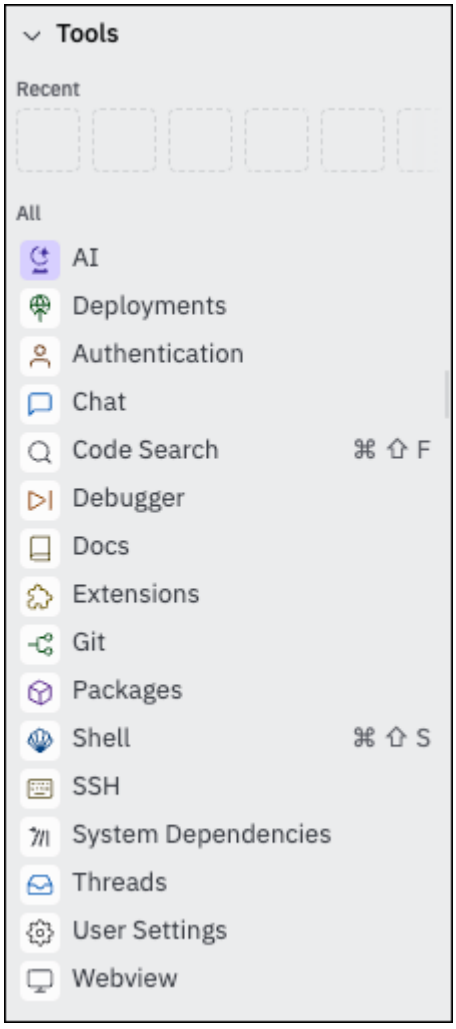
Tabs and panes

To the right of the sidebar are panes. In each pane, there may be multiple open tabs. When you first open the workspace, you'll see two panes. The

first displays a code editor, and the second contains two tabs: Webview and Shell.

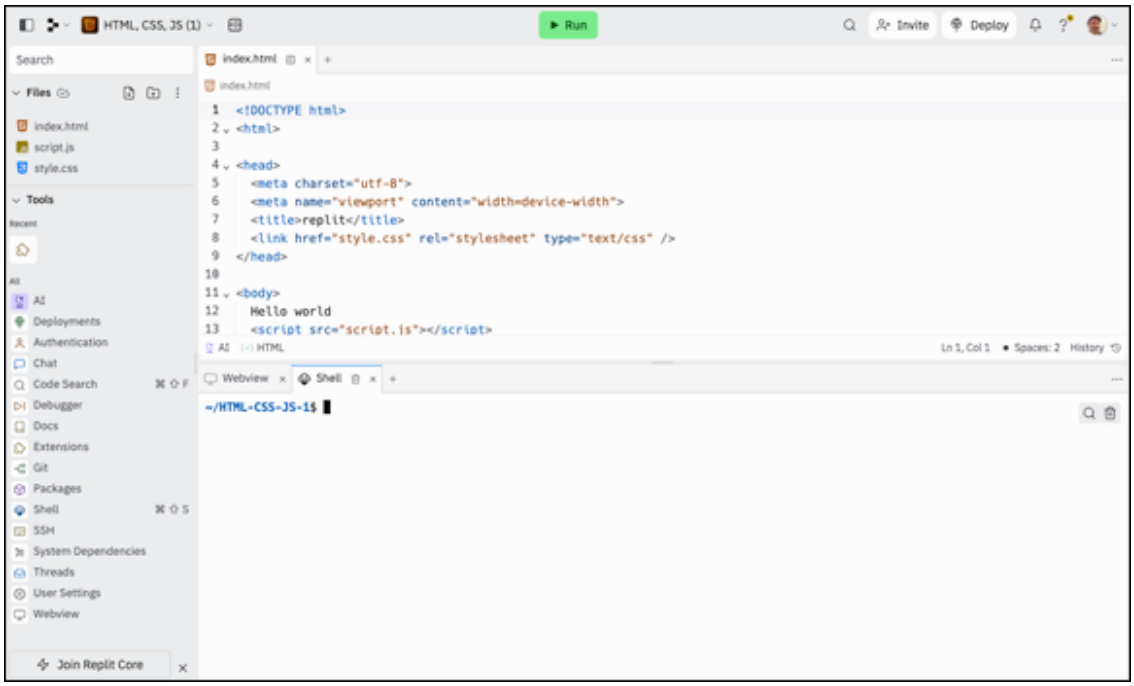
The Webview tool displays a live preview of your project as it appears in a web browser. The shell tool provides access to a Linux command shell.

You can rearrange tabs and panes in the workspace by clicking and dragging their headers. For example, in Figure 2-15, I moved the pane containing the Webview and Shell tabs to the bottom of the browser window.



Replit, Inc.

FIGURE 2-14: Viewing the workspace tools panel.

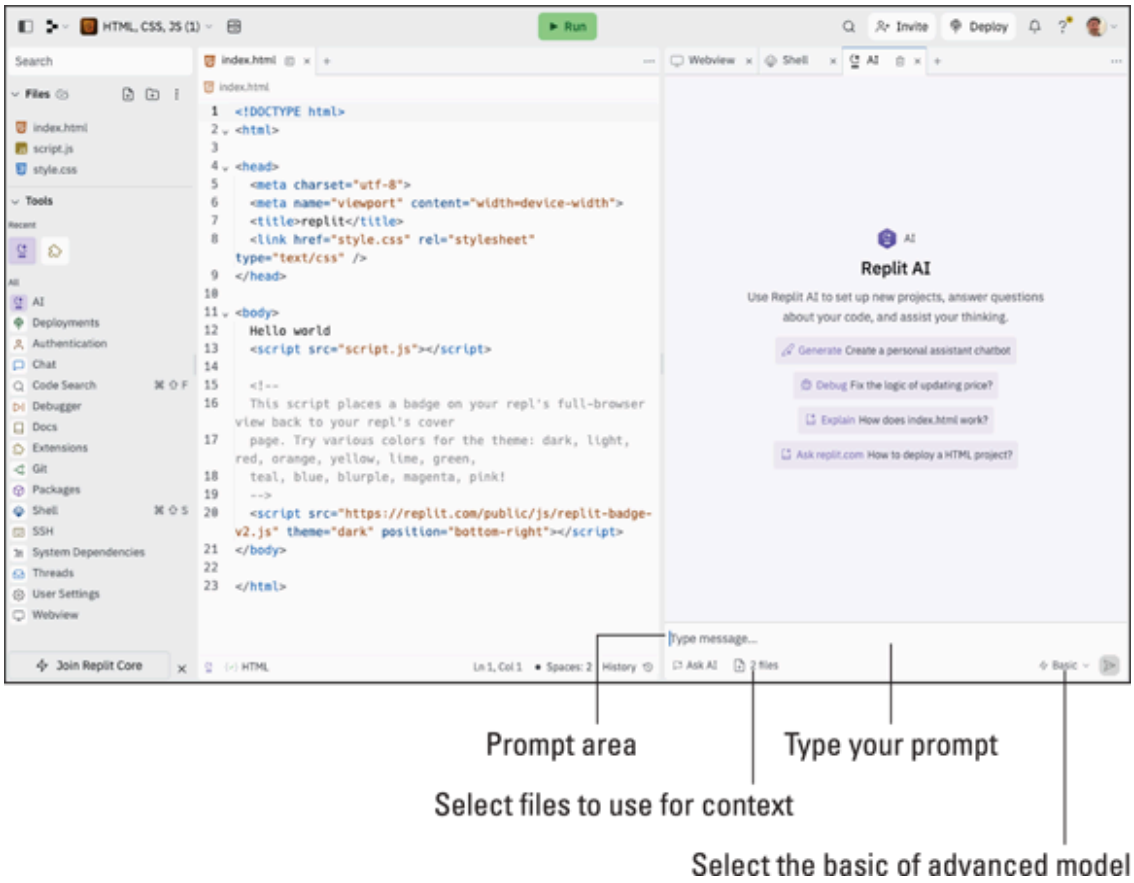


Replit, Inc.

FIGURE 2-15: Rearranging panes.

Pairing up with Replit AI

Replit's AI assistant is called Replit AI. With the workspace open, you'll see a link to AI in the Tools panel at the lower left of the screen (refer to Figure 2-15). Clicking the AI icon opens a new tab to the right of the code editor, as shown in Figure 2-16.



Replit, Inc.

FIGURE 2-16: The AI tab opens in the right panel.

Replit AI is available to all Replit members. However, paid Replit members can access a more advanced version of the AI by going to `https://replit.com/cycles`, where you can also see your balance of Cycles (Replit's virtual tokens) and buy Cycles.



REMEMBER You can earn Cycles by completing Bounties.

Replit AI has four tools: Generate, Debug, Explain, and Modify, in addition to Copilot-style code completion as you code. To access the Generate, Debug, and Explain tools, right-click in the code editor. The Modify and Explain tools, which change and explain code, respectively, are available only when you've selected code you want the AI to modify or explain.

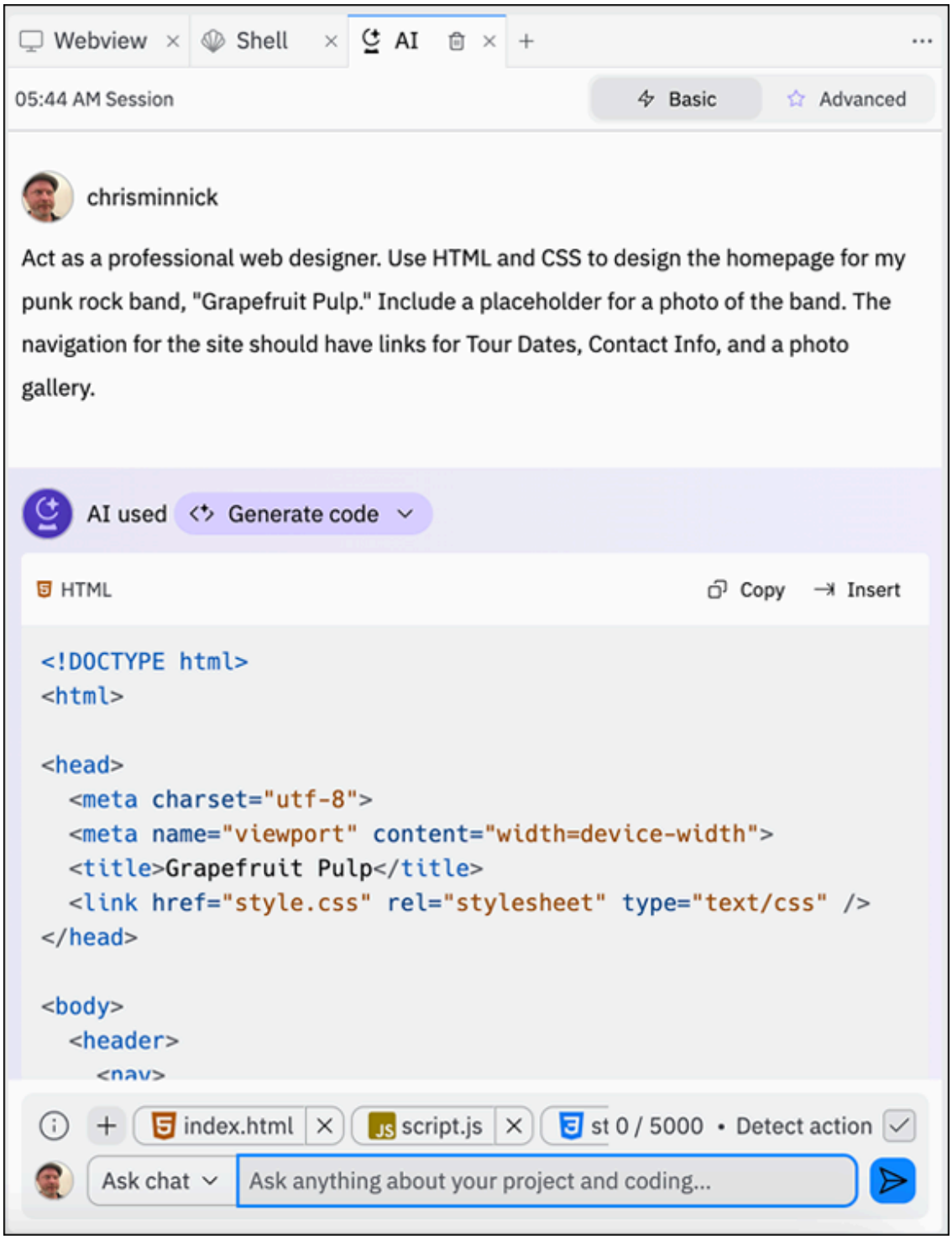
All of the tools can be used through the Chat interface as well. At the bottom of the AI pane is the text box for prompting the model. Follow these

steps to begin using Replit to build a website for a fictitious punk rock band called Grapefruit Pulp:

1. **Give AI the following prompt:**
   ```
   Act as a professional web designer. Use HTML and CSS to design the homepage for my punk rock band, "Grapefruit Pulp." Include a placeholder for a photo of the band. The navigation for the site should have links for Tour Dates, Contact Info, and a photo gallery.
   ```
   After a moment, the model returns some HTML and CSS code. My results are shown in Figure 2-17.
2. **Select everything in index.html and delete the sample template code.**
3. **Click the Insert link at the top of the HTML response to insert the generated HTML into index.html.**
   The chat interface can't interact with your files directly, so you need to copy any suggestions that you like from the chat window to your files yourself (using copy and paste or the insert link).
4. **Open style.css and delete its contents.**
5. **Use the Insert link at the top of the CSS response to insert the CSS into style.css.**
6. **Click the Run button at the top of the IDE to open your website in Replit's web view.**
   Figure 2-18 shows my site. It isn't spectacular, but it's a decent start.

Replit AI has two different models: basic and advanced. The basic model is faster but more limited. The advanced model is larger and slower but much more likely to give a better response to a vague or creative prompt. The advanced model is available only with a paid Replit account.

If you have a paid account, you can retry the prompt with the advanced model by toggling the Basic/Advanced selection at the bottom of the AI window (labeled in Figure 2-16). Or, if you're a paid member, you can use the advanced model by default by clicking the Advanced button at the top of the AI chat interface (refer to Figure 2-17).



Replit, Inc.

FIGURE 2-17: Generated HTML and CSS from Replit AI.



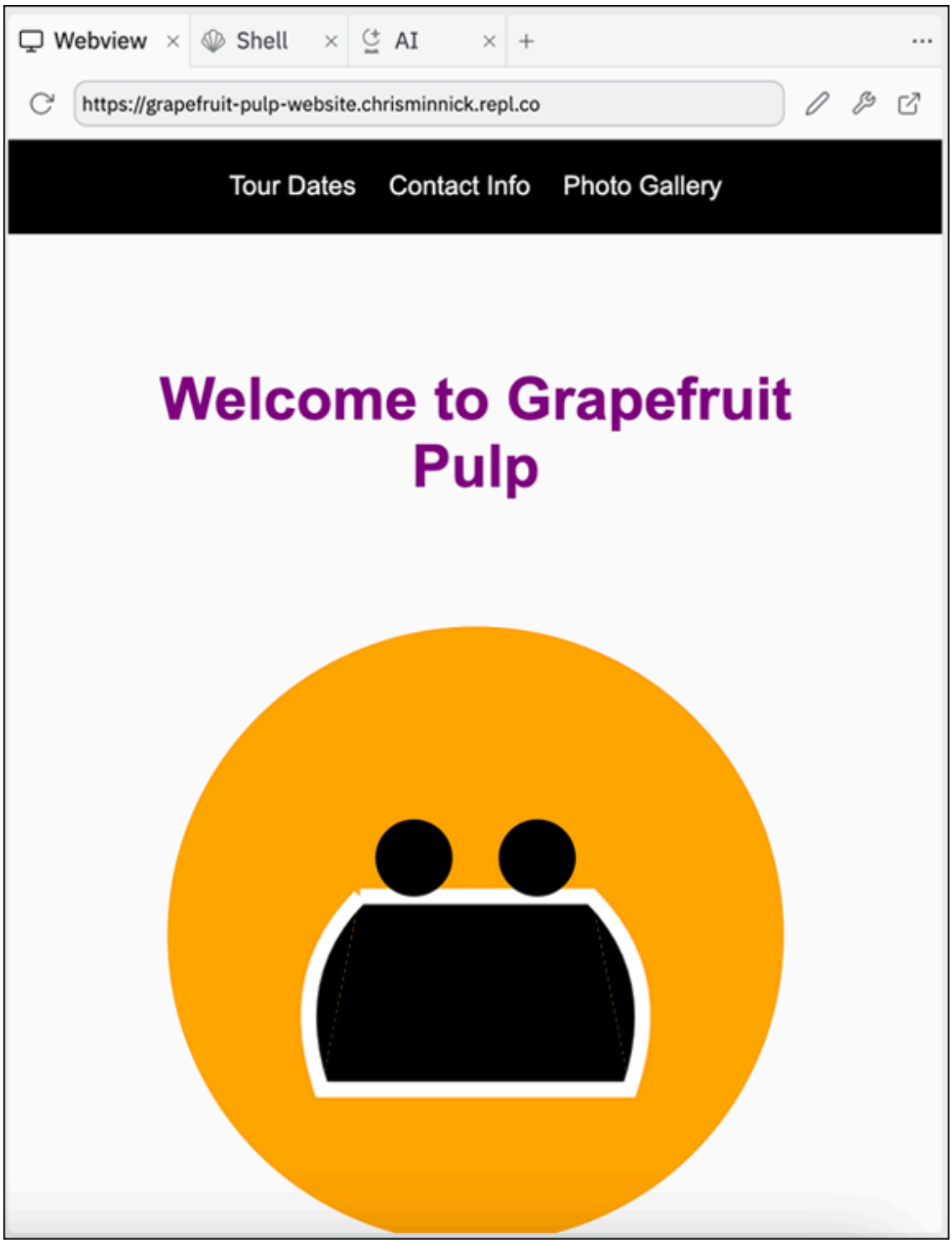
Replit, Inc.

FIGURE 2-18: The start of my website for Grapefruit Pulp.

To see how good Replit AI's advanced model is with more difficult tasks, I gave it the following prompt:

```
Use SVG and CSS to draw a grapefruit wearing a leather
jacket. It should be inserted in place of the placeholder
image.
```

The result is shown in Figure 2-19. This prompt seemed to push the limits of the model's creativity, although this might be a start for an awesome logo.



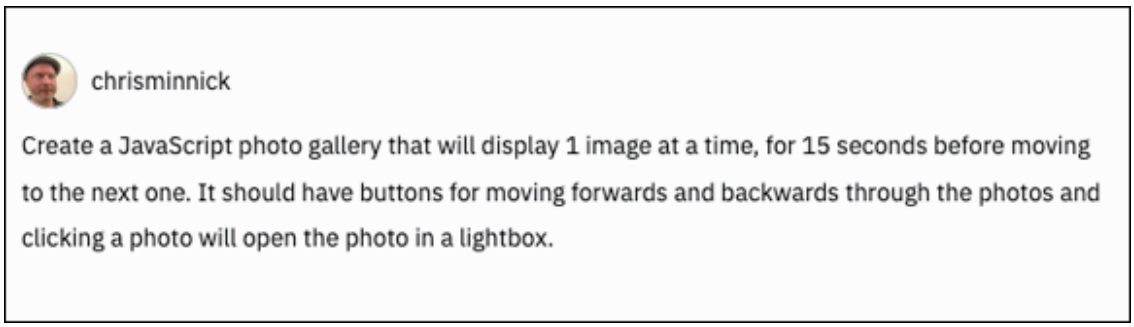
Replit, Inc.

FIGURE 2-19: Replit AI's SVG punk grapefruit.

Next, I switched to working in the photo gallery. I gave the model the prompt shown in Figure 2-20.

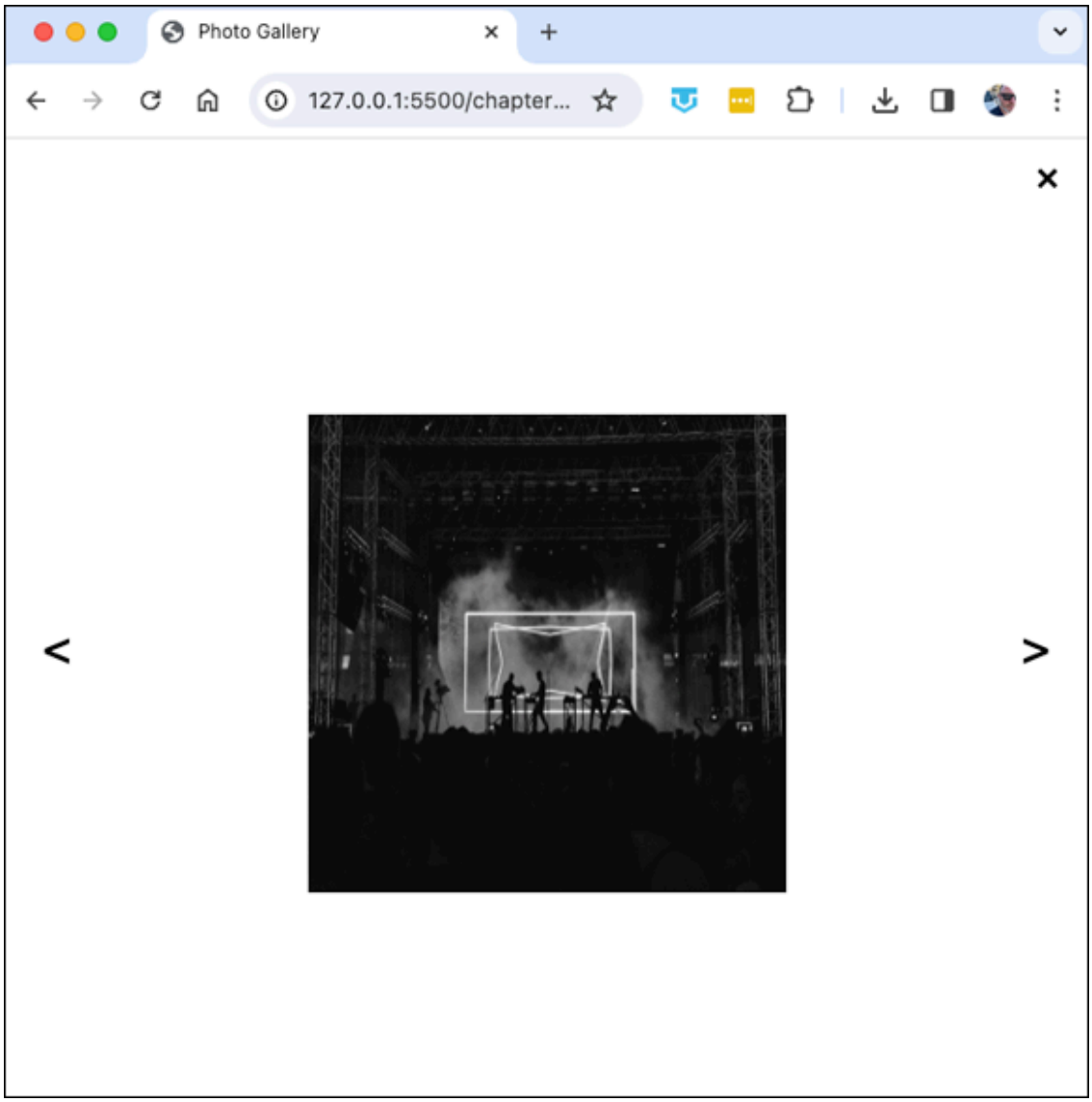
When I uploaded some images, the result was decent and close to what I had imagined. Figure 2-21 shows the photo gallery after I clicked a photo to open it in a lightbox.

Used correctly, coding assistant tools such as the ones you learned about in this chapter can make you a more productive coder. In the next chapter, you discover how to talk to AI chat tools to generate ideas and code.



Replit, Inc.

FIGURE 2-20: Prompting for a JavaScript photo gallery.



From Yannis Papanastasopoulos

FIGURE 2-21: My Replit AI-generated lightbox.